Technical report — NLI over Swiss federal voting booklets with Apertus

What we built, how it works, and what the numbers say. Every number comes from a recorded run in `docs/runs/` (short names in brackets, listed under References) on the dev split, the validation set (val), the probe set or all three; **the sealed test dates have not been run**.

- **Track:** `Track 2A — OST: Multilingual Natural Language Inference over Swiss Official Voting Booklets`
- **Event:** Online
- **Project:** TODO project name
- **Team:** TODO team name — Rashad, Kaan (TODO full names)
- **Demo:** none

1. Summary

Given a claim in German, French or Italian and a source from an official Swiss federal voting booklet, the system decides whether the source supports (entailment, 0), does not settle (neutral, 1) or refutes (contradiction, 2) the claim. The source is a passage (task B) or a whole booklet PDF (task A). **Apertus v1.5 makes every decision.** Everything else is local and runs on CPU in the Docker image: reading the PDF, choosing the part of the booklet Apertus reads, and turning the paragraphs Apertus cites into evidence quoted verbatim with their 1-based PDF page. Apertus also names the part a claim refers to when the rules do not know the claim's opening (section 4).

The central experiment, full booklet against selected context, decided the design: in one interleaved run on 100 dev cases [E1-curve] the whole booklet scored 0.858 with 37,586 input tokens per case, the part of the vote that the claim names (`section-route`) 0.989 with 1,248; without any booklet text Apertus scores 0.435 [E2-cb].

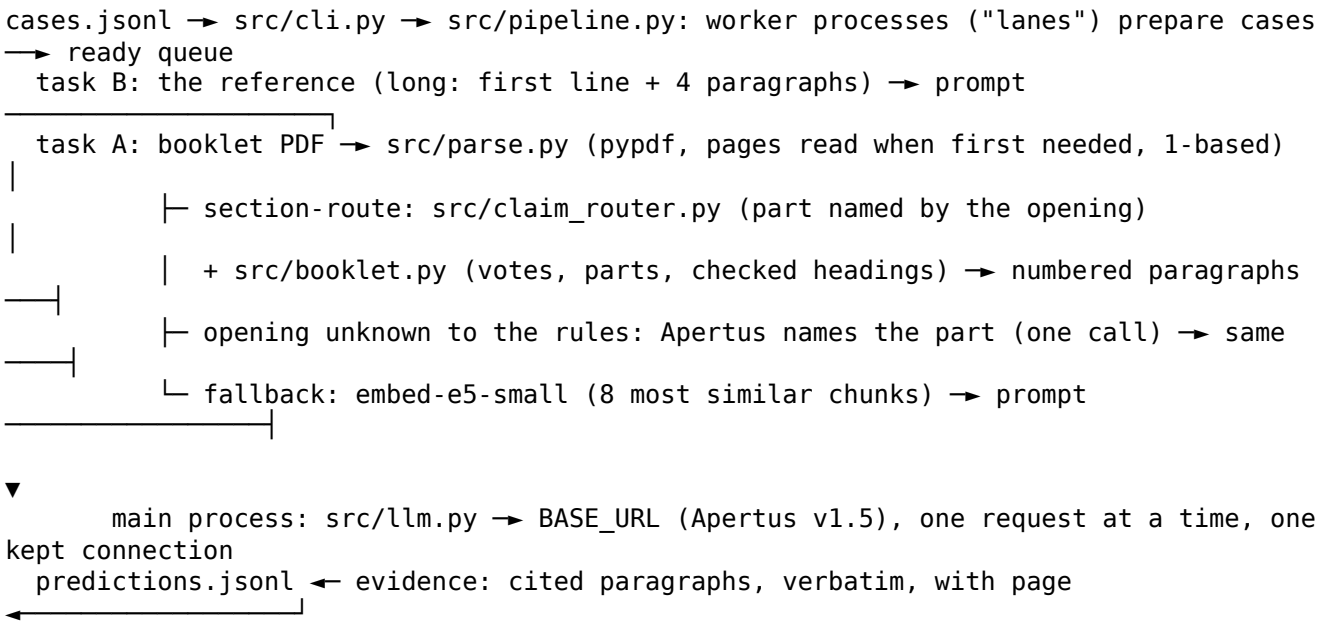
Final defaults, published image [FIN]	dev	val	probe (2 unseen dates)
Task A Macro-F1 (weight 0.30 of the score)	**0.980**	**0.959**	**0.971**
Task A evidence (0.12)	**0.985**	**0.955**	**0.954**
Task A tokens per case, input + output (K, 0.09)	1,238 + 14.5	1,206 + 14.5	1,136 + 14.5
Task A local time per case, model at 1.0 / 0.3 s per request (T, 0.09)	0.018 / 0.155 s (repeat: 0.108)	0.008 / 0.010 s	0.026 / 0.237 s
Task B Macro-F1 (0.28)	**0.987**	**0.968**	**0.977**
Task B tokens per case, input + output (K, 0.06)	815 + 8.7	820 + 9.3	789 + 8.9
Task B local time per case, 1.0 / 0.3 s per request (T, 0.06)	0.010 / 0.009 s	0.007 / 0.007 s	0.015 / 0.015 s
Labels written by Python (neutral fallback)	0 of 600	1 of 1,160	0 of 188
REF SCORE at 1.0 / 0.3 s per request (section 2)	**0.909 / 0.851**	**0.908 / 0.906**	**0.897 / 0.825**

Sealed test dates (2020-02-09, 2022-02-13, 2022-05-15): TODO, results after the organisers' evaluation.

2. What is scored, and what we optimise

The organisers' score (Q&A of 8 October; `docs/official_scoring.md`): **Score = 0.4 S_B + 0.6 S_A**, with S_A = 0.50 Macro-F1 + 0.20 evidence + 0.15 T + 0.15 K and S_B = 0.70 Macro-F1 + 0.15 T + 0.15 K. K = the best team's tokens per case ÷ ours; T = the best local time per case ÷ ours, where **local time is the container's wall-clock time minus the time during which a model request is in flight**: start-up, PDF reading, embedding and every gap between two requests count. Task A and task B run in separate containers. To rank our own versions we use **REF SCORE** (`scripts/official_score.py`): the same formula with "best" values fixed by us before any measurement (600 / 800 tokens, 0.05 / 0.005 s local time per case for task A / B). Local time is measured the organisers' way (`scripts/local_time.py`): the image runs the organisers' command with 2 CPUs and 4 GB against a fake model that replays saved real answers after a fixed delay, 1.0 s (about Public AI's speed), 0.3 s or 0.1 s (fast servers), and logs when each request is in flight.

3. Architecture



- `src/cli.py`: the official entrypoint. Each case is split into `prepare()` (all local work) and `complete()` (the model calls and the response). Answers every request that has an id, never stops on one bad case (fallback: neutral, no evidence), writes each answer at once, exit code 0.
- `src/pipeline.py`: two forked lanes prepare the cases while the main process sends the prepared ones. Each lane owns whole booklets and first prepares the cases that need no embedding model; the lanes split the CPUs for onnxruntime; under a 2.5 GiB memory limit one lane. Results do not depend on order, timing or memory: every cache is keyed by content, and a check replays the saved answers of all 1,180 dev and val requests and finds every request, label and evidence unchanged (G1, G2).
- `src/llm.py`: the only module that calls a model (`chat/completions` at `BASE_URL`, temperature 0, one kept connection, retries for 5xx, timeouts and 429).

3.1 How document context is prepared and supplied to the model

Parsing. pypdf reads a page's text the first time it is needed and keeps its 1-based PDF page number (the *n*-th page of the file); a page whose fonts pypdf cannot read is repaired (two embedded font programs in one font descriptor) or left empty, never failing the booklet.

Selection (`section-route`, task A). (1) *Router:* the claims open with a formula naming their source ("Laut der Zusammenfassung …", "Le Conseil fédéral estime …"); regular expressions map it to one of five parts: summary, Federal Council, committee, text put to the vote, detailed explanation. If they find none, Apertus names the part (one call). (2) *Parser:* the table of contents gives each vote's sections (two contents layouts: before and after mid-2018); every start page is checked against its printed heading; the arguments are split by voice; a part that passes its checks

is used even if another part of its vote fails. (3) *Vote match:* the case's `vote` against the contents titles (fuzzy); if that fails, by meaning with multilingual-e5-small. (4) *Paragraphs:* the part's lines grouped into paragraphs, each with its page; a part over 8,000 characters keeps its 8 paragraphs most similar to the claim (e5). (5) *Fallback:* no part, no vote, or any error → the 8 chunks of the booklet most similar to the claim.

What the prompt contains. System: the instruction and the decision rule (`A-v4-section-route-L1`):

> *Decide in this order: 1. Does the reference text deal with the subject of the claim at all? If not, the label is 1 (neutral). 2. If it does: 0 if it supports the claim; 2 only if it states something that cannot be true together with the claim; otherwise 1. Missing information is never a contradiction. […]*

User: `PART:` (which part, whose voice), `REFERENCE TEXT:` with paragraphs `[1] … [n]`, `VOTE:`, `CLAIM:`. Nothing is translated. `response_format: json_schema` forces `{"paragraphs": [≤ 3 numbers], "label": 0|1|2}`. Task B sends the same rule (`v3-topic-first`) and asks for `{"label": n}`; a reference over 3,000 characters is sent as its first line and the 4 paragraphs whose first 300 characters are most similar to the claim (e5).

Evidence. Only what Apertus cited: the cited paragraphs verbatim with their page, then the two halves of each (cut at the sentence end nearest the middle) up to five items. Neutral answers and task B carry none.

3.2 Who does what

Step	Python (rules, deterministic)	Local model (multilingual-e5-small, CPU)	Apertus v1.5
Read input, answer every id; PDF text per page	✓ (pypdf)		
Which part the claim names	✓ (regular expressions)		only when the rules find none
Which vote; where its parts start	✓ (contents, checked headings)	vote by meaning if the title match fails	(tried, not used: section 4)
Which paragraphs	✓ (the part)	parts over 8,000 characters; fallback; task B references over 3,000 characters	
Label	only the neutral fallback for a failed call or an unreadable answer: 1 of 2,128 final labels (a val task B answer), 0 in task A		✓ **every label**
Which paragraphs justify it; evidence	copies what Apertus cited, with its page		✓ cites them

4. Use of Apertus

- **Model:** Apertus v1.5, named by `MODEL` (default `swiss-ai/Apertus-v1.5-8B`). Our runs used `swiss-ai/apertus-v1.5-8b` on Public AI. Which model and server the evaluation uses is not known to us.
- **How:** inference only, zero-shot, temperature 0; one call per case, plus one when the rules do not know a claim's opening; answer budget 128 tokens (task A), 32 (task B). No fine-tuning. Reached only through `BASE_URL`.
- **Apertus as parser** (session 11). We tried Apertus in the two roles Python plays before the answer; every parsing call counts in the case's tokens, a booklet's call in its first case:

Role	Accuracy	Effect (interleaved runs)	Tokens per case, input + output	In the final system?
AP-part: name the part a claim's opening refers to (one call per claim)	79–85 % of reworded openings (council, summary, committee 95–100 %; law about 50 %; detail 16–24 %)	claims with openings the rules do not know (150 reworded per set): Macro-F1 0.967 / 0.926 (dev / val) against 0.894 / 0.876 for the best fallback, evidence 0.880 / 0.840 against 0.670 / 0.650	+236 + 49 on such cases	**yes, after the rules** (≥ +0.02 on dev and val); no dev, val or probe claim needs it (the rules name a part for all)
AP-structure: read the contents (one call per booklet)	all four start pages equal to the parser's for 78.7 % of votes the parser passes (93 booklets)	with the parser switched off (dev): 0.970 / 0.955 against 0.886 / 0.687	1,796 + 176 per booklet (≈ 268 + 29 per case)	no: below the 95 % agreement required
Both for every case (AP-all)	part as the rules' 89 %; vote placed 93 %	dev 0.960 / 0.891 against 0.977 / 0.980; probe 0.951 / 0.877 against 0.971 / 0.954	+502 + 77 on dev	no (reported only); local time 0.822 s per case against 0.101 s at 0.3 s per request
Read a page image	the endpoint accepts it, the answer does not name the page's heading	—	1,077 + 26	no

- **Development tools:** Claude Code wrote code and ran experiments; it is never called by the pipeline.

5. Data

- **Dataset:** `OSTswiss/MNLIoverSwissVotingBooklets` v1.1 (commit `9ff08597`, MIT): 1,488 rows, 335 exact duplicates; each row gives one task A and one task B case.
- **Splits** (by voting date, duplicates removed): test 5 dates (267 rows); dev 300 rows; val 580 rows of the dev dates (never used to change a rule); **probe:** the 94 test rows of 2021-03-07 and 2025-09-28; three dates sealed.
- **Booklets:** the Federal Chancellery's PDFs, downloaded by script and not committed. For parser coverage, every federal booklet from 2016 to 2026 in de, fr and it (93 PDFs; the sealed dates never downloaded). Swiss copyright law exempts official reports of authorities (Art. 5 URG). No personal data.

6. Evaluation

Method. The starter's `evaluate.py`, unchanged: Macro-F1 per task; for task A the share of gold entailment/contradiction cases where one of the first five evidence items matches the gold passage. The endpoint drifts, so **only arms of one interleaved run compare** (every case answered by all versions back to back; all session 11 answers came from one backend). Rules were fixed before the runs; a version that fails stays in the code, switched off. Changes that should not change answers are adopted only if every request, label and evidence of the 1,180 dev and val cases stays identical (G1, G2) and the measure improves.

Parser coverage (93 booklets 2016–2026; no labels needed: the three languages of a date must agree) [COV]: booklets whose votes all pass 64 → **88**; votes passing every check 178 → **244 of**

249 (all 249 with a usable part); dates where the languages disagree 2 → 0; probe cases routed 83 → **90 of 94**. Fixed, most frequent first: the contents layout used until mid-2018 (20 booklets), unreadable fonts (7), a lost heading, one failed part failing its whole vote.

Fallback levels, forced [FB-dev, FB-val]: every case put into a fallback situation by a test-only switch (“no part”: the router returns nothing; “no vote”: the vote match fails). Macro-F1 / evidence, dev; val:

Level	dev	val (balanced 300)	Input tokens	Adopted?
routed (today, not forced)	0.977 / 0.980	—	1,238	
embed-e5-small, 8 chunks of the booklet (the fallback)	0.900 / 0.672	0.875 / 0.593	1,868	stays
vote-wide: the vote’s paragraphs, part named	0.927 / 0.766	0.873 / 0.716	1,078	no (val −0.002, needs +0.02)
booklet-wide: the booklet’s paragraphs	0.857 / 0.647	0.856 / 0.627	939	no (lower)

Task B, fewer tokens [TB-dev, TB-val]: a reference over 3,000 characters is cut to its first line (the ballot’s title) and the 4 paragraphs most similar to the claim. Embedding whole paragraphs (E) costs local time (0.661 s per case at 0.3 s per request); comparing only each paragraph’s first 300 characters (T) costs 0.0089 s (the whole reference: 0.0066 s). Task B Macro-F1, dev; val: whole reference 0.967 (2 unreadable); 0.954 (3) with 1,994 input tokens per case; **T 0.987 (0); 0.968 (1) with 815** (−59 %); E 0.987 on dev. T is the default.

Local time (the organisers’ measure; image, 2 CPUs, fake model replaying the final answers) [FIN, STEP4]. A profile at 0.3 s per request showed the two lanes busy all the time: PDF reading (about 1.3 s per booklet, pure-Python pypdf) and e5 embedding of long parts (about 3 s each), and two lanes each running onnxruntime with every CPU. Three changes that keep every request and answer identical (G1, G2): cases needing no embedding first, pages read when first needed, onnxruntime threads shared between lanes: task A 0.202 → 0.101 s per case at 0.3 s per request.

Final image, dev, local time per case	1.0 s per request	0.3 s	0.1 s
Task A	0.018 s	0.155 s (repeat 0.108 s)	0.316 s
Task B	0.010 s	0.009 s	0.130 s

Measured the same way and on the same CPUs, session 10’s image needs 0.223 s per case for task A at 0.3 s and this image 0.129 s; single measurements vary by up to ±0.03 s. What is left is the time to the first request (about 3.5 s: container, Python, the first booklet), PDF reading and long parts’ embedding, which with a fast model no longer hide behind the requests.

Memory [MEM]: each lane peaks at about 1.2 GiB, almost all of it loading e5; two lanes needed 3 GiB. Under a 2.5 GiB limit the image now runs one lane: every response, with the same labels, at 2 GiB and 1.5 GiB (2 CPUs); 1 GiB fails (e5’s load peak). Task B needs under 0.1 GB. On the final image, task B (whose selector loads e5 for long references) also needs 1.5 GiB.

Versions tried in session 11 (each by a rule fixed before its run; a version that fails stays in the code, off):

Version	Rule, outcome
Parser fixes; light-first, lazy pages, shared CPUs; one lane under 2.5 GiB	same answers (G1, G2), the measure improves: **adopted**
AP-part after the rules; task B selector T	their rules on dev and val: **adopted**

Version	Rule, outcome
Vote-wide / booklet-wide fallback; AP-structure; quantised e5 (task A); E, Q (task B); arena off	fail their rules: off

Probe: two unseen voting dates (test rows of 2021-03-07 and 2025-09-28, opened in session 10). Through the final image [FIN-probe]: task A **0.971**, evidence **0.954** (62 of 65), task B **0.977**; REF SCORE 0.897 / 0.825. Session 10's image: 0.960, 0.908, 0.971. 90 of 94 task A cases are routed (83 before step 1's parser fixes); the 4 others are claims about a committee in votes that have none, answered by the fallback.

7. Limitations

- **Tuned on dev.** Prompts, router patterns and parser checks were written with dev in view; val confirms; the probe (two unseen dates) lands at val's level; three test dates and the private set are unseen.
- **`section-route` depends on the booklet layout** (2016–2026 checked: 244 of 249 votes pass). A claim whose opening neither the rules nor Apertus place, or a vote the parser cannot read, falls back (answered, less accurate).
- **Endpoint and speed.** Our scores hold for the backend that answered (Public AI served two); with a fast model, PDF reading and embedding no longer hide behind the requests.
- **Not political advice.** The system checks a claim against the booklet's text only. Its answers are not voting recommendations. Every task A label 0 or 2 comes with the booklet page and the verbatim text it rests on.

8. Reproducibility

- **Published image:** `docker pull ghcr.io/rashad18236/hackapertus-voting-nli:latest` (built from `main` by `.github/workflows/publish-image.yml`, which also runs it on a fresh runner without internet); then the organisers' command with `BASE_URL` and `API_KEY` (README). This report's final numbers come from `sha-bb2d08fea0f1` (the `session-11` tag of the branch).
- **From source:** `make run` builds the image and answers `examples/cases.jsonl`.
- **Hardware:** CPU only; 2 CPUs and 4 GB measured (peak 2.3 GiB with two worker processes); the image also answers every case with 1.5 GiB (one worker process under 2.5 GiB) [MEM].
- **Determinism:** temperature 0; a fake model replays saved answers by request hash (G1, G2); 273 tests.

License

Code: Apache-2.0 (the repository's `LICENSE`). Report text: Creative Commons Attribution 4.0 (CC-BY-4.0).

References

- Dataset: OSTswiss/MNLIoverSwissVotingBooklets v1.1, Hugging Face (MIT). Starter: `gitlab.com/ifsoftware/hackapertus-starter`. Apertus v1.5, Swiss AI Initiative.
- Wang, L. et al. (2024). *Multilingual E5 Text Embeddings: A Technical Report.* arXiv:2402.05672 (`intfloat/multilingual-e5-small`, MIT). pypdf (BSD-3), rapidfuzz (MIT), onnxruntime (MIT), requests (Apache-2.0).
- Runs (`docs/runs/` + name): FIN `2026-10-13_rashad_final-s11_{dev600,val1160}`, FIN-probe `2026-10-13_rashad_final-s11_probe188`; STEP4 `2026-10-12_rashad_local-time-step4_devA300`; MEM `2026-10-13_rashad_memory-step5_devA300`; TB-dev / TB-val `2026-10-13_rashad_taskb-selectors_{devB300,valB580}`; AP runs `2026-10-1*_rashad_ap-*`; replay reference `2026-10-13_rashad_prompt-snapshot-final-s11_devAB-valA`; COV `2026-10-12_rashad_parser-coverage_booklets93`; FB-dev / FB-val `2026-10-12_rashad_fallbacks_{devA300,valA300}`; from 2026-10-09: E1-curve `context-curve_devA100`, E2-cb `closed-book_devA300`.